

# Advanced AI for IT

Day 2 | Own the model

OQ | Muscat | Monday 28 September 2026

**Facilitator: Ritesh Vajariya**

DAY 2 OF 5

# Own the model

By 16:00 you will have **served a model yourself, measured it under load, and fine-tuned it** on service desk tickets.

Day 1 Architect

Day 2 Own the model

Day 3 Ground it

Day 4 Agents

Day 5 Integrate and ship

DAY 2 | AGENDA

# Four labs today, with the talks kept short and a buffer for the GPU

|       |     |                                            |              |        |
|-------|-----|--------------------------------------------|--------------|--------|
| 08:00 |     | Day 1 recap: your spec v0.2 in two minutes | Discussion   | 15 min |
| 08:15 | S6  | Why own the model                          | Talk         | 15 min |
| 08:30 | S7  | Run a model yourself                       | Lab          | 60 min |
| 09:30 | S8  | What breaks at scale                       | Lab and talk | 45 min |
| 10:15 |     | <b>Break</b>                               |              | 15 min |
| 10:30 | S9  | When to fine-tune                          | Talk         | 30 min |
| 11:00 | S10 | Build the dataset                          | Lab          | 50 min |
| 11:50 |     | <b>Dhuhr and lunch</b>                     |              | 55 min |
| 12:45 | S11 | Fine-tune it                               | Lab          | 90 min |
| 14:15 |     | <b>Break</b>                               |              | 15 min |
| 14:30 | S12 | Did it work                                | Lab          | 45 min |
| 15:15 |     | <b>Break and Asr</b>                       |              | 15 min |
| 15:30 |     | Case study: the model that got worse       | Discussion   | 15 min |
| 15:45 |     | Close: spec v0.3 and tomorrow              | Facilitated  | 15 min |

DAY 2 | BRIDGE

# Yesterday you chose where a model runs. Today you run one

## WHAT DAY 1 PRODUCED

**Your hosting decision**

**Your cost at 1K and 100K queries**

**Your capstone group and use case**

## WHAT TODAY DOES WITH IT

S7: you serve a model on a machine you control

S8: you price self-hosting against that API figure

S9 to S12: you decide whether tuning belongs in it

DAY 2 | S6 WHY OWN THE MODEL

# Five reasons OQ would run its own model

1

## Classification

Restricted data never leaves a machine you control

2

## Residency

The model runs in the country and region you choose

3

## Lock-in

No single vendor can reprice or withdraw your model

4

## Cost at volume

Past a steady load, a busy server beats per-token pricing

5

## Version stability

The model changes only when you decide it changes

Yesterday's spec already told you which of these apply to your system.

DAY 2 | S6 WHY OWN THE MODEL

# The honest cost of ownership: hardware, operations, evaluation and on-call

1

## Hardware

GPUs sized for your peak, not your average

2

## Operations

Updates, patching, monitoring, capacity

3

## Evaluation

Every new model or version re-tested before release

4

## On-call

Someone answers when it stops at 03:00

If nobody would own the on-call rota, the answer is managed cloud.

DAY 2 | S6 WHY OWN THE MODEL

**Self-hosted means a machine you control, not a laptop. Today Colab stands in for OQ's own VM**

**A laptop**

Fine for learning. Not a production host

**Today: a Colab runtime**

A cloud VM we control for the day

**At OQ: your own cloud VM**

Same software, OQ's tenancy and region

**The lesson is self-hosted against vendor API, not laptop against cloud.**

DAY 2 | S6 WHY OWN THE MODEL

# Local and hosted are one line of code apart

## YESTERDAY'S NOTEBOOK

```
client = OpenAI(base_url="https://api.provider.example/v1", api_key=KEY)
```

## TODAY, POINTED AT YOUR OWN MODEL SERVER

```
client = OpenAI(base_url="http://localhost:11434/v1", api_key="unused")
```

**Most local servers speak the same API shape as the hosted ones. Your prompts, parsing and tests stay as they are.**

Read: [Ollama OpenAI-compatible API: docs.ollama.com](https://docs.ollama.com)

DAY 2 | S6 WHY OWN THE MODEL

# Today's plan: serve it, stress it, tune it, test it



DAY 2 | S7 RUN A MODEL YOURSELF | LAB

# Serve a small model on a machine you control, and call it from yesterday's code

NOTEBOOK

**02\_local\_inference**

TIME BOX

50 minutes, 40 in the notebook

RUNS ON

Colab. Runtime type is in the notebook header

WORK IN

Your capstone group

IN AZURE

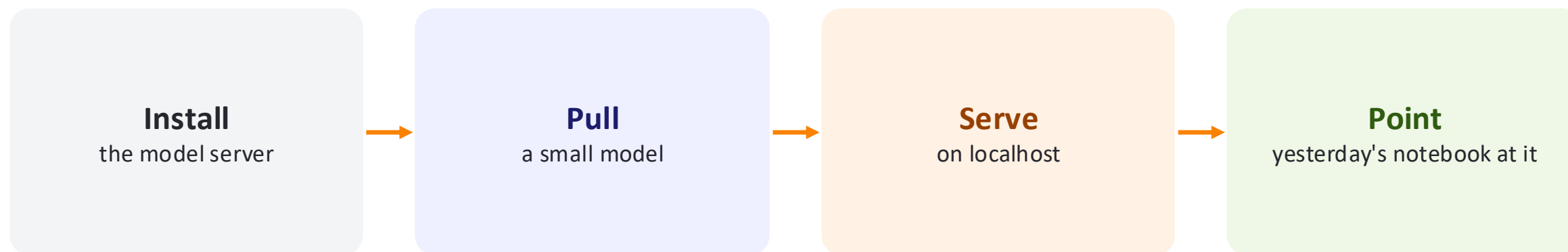
A GPU VM you run in your tenant (NVadsA10\_v5, NCasT4\_v3), or an Azure ML online endpoint



Scan to open

Help: [Ollama FAQ: docs.ollama.com/faq](https://docs.ollama.com/faq) | [Google Colab FAQ: research.google.com/colaboratory](https://research.google.com/colaboratory)

## You are building your own model endpoint



**The only change to yesterday's code:** the base URL.

## Four steps, and the last one changes one line

1

### Install

Run the install cell and wait for the server version to print

2

### Pull

Pull the small model named in the notebook

3

### Serve

Start the server and send one test prompt

4

### Point

Change the base URL in yesterday's notebook and rerun it

DAY 2 | S7 RUN A MODEL YOURSELF | LAB

# Checkpoint: the same prompt, answered by your own endpoint

## YOU SHOULD NOW SEE

✓ A server version printed

✓ One test answer from localhost

✓ Yesterday's notebook runs with one line changed

### MEASURED ON A COLAB T4, 23 SEPT

```
server      Ollama 0.12.10, installed in 112 s
model       llama3.2:1b, 100% in GPU memory
speed       158.6 tokens per second
```

**ticket reply from your endpoint: not JSON**  
same ticket, vendor model: 4 of 6 fields

LOCAL INFERENCE READY

Owning the model does not make it good at the task.  
That is what the afternoon is for.

DAY 2 | S7 RUN A MODEL YOURSELF | DEBRIEF

## Switching to your own model is an operations change, not a code change

How many lines of your code changed?

**This feeds S8 at 09:30.** Next we find out what your endpoint does with sixteen users instead of one.

DAY 2 | S8 WHAT BREAKS AT SCALE | LAB

# Hit your own endpoint with one user, then sixteen, and time every answer

NOTEBOOK

**03\_concurrency**

TIME BOX

15 minutes in the notebook, then talk

RUNS ON

The endpoint from S7

WORK IN

Your capstone group

IN AZURE

The same test against your own Azure endpoint. Production serving: vLLM on that VM



Scan to open

DAY 2 | S8 WHAT BREAKS AT SCALE | LAB

# Checkpoint: response time from 1 to 16 users, and your capacity at the target

YOU SHOULD NOW SEE

✓ p95 response time at each level

✓ Throughput flattening out

✓ Capacity at your latency target

**MEASURED ON A COLAB T4, 23 SEPT**

llama3.2:1b, real tickets, 80 requests, 0 failed

| callers | p95 | seconds |
|---------|-----|---------|
|---------|-----|---------|

|   |      |  |
|---|------|--|
| 1 | 1.46 |  |
|---|------|--|

|    |       |             |
|----|-------|-------------|
| 16 | 11.73 | (8x slower) |
|----|-------|-------------|

throughput flat at about 1.4 requests/s

capacity at p95 &lt;= 10 s:

8 callers, 1.41 req/s, 5,076 req/hour

Your numbers will differ. The shape will not.

DAY 2 | S8 WHAT BREAKS AT SCALE | DEBRIEF

## Your month-end peak decides the hardware, not your average day

Where would OQ's month-end peak put you on this table?

**The talk starts now.** Why it slows down, and what production serving does about it.

DAY 2 | S8 WHAT BREAKS AT SCALE

## One user is a demo, sixteen users is a system

1 user

**1.46 s**

p95 response time

16 users

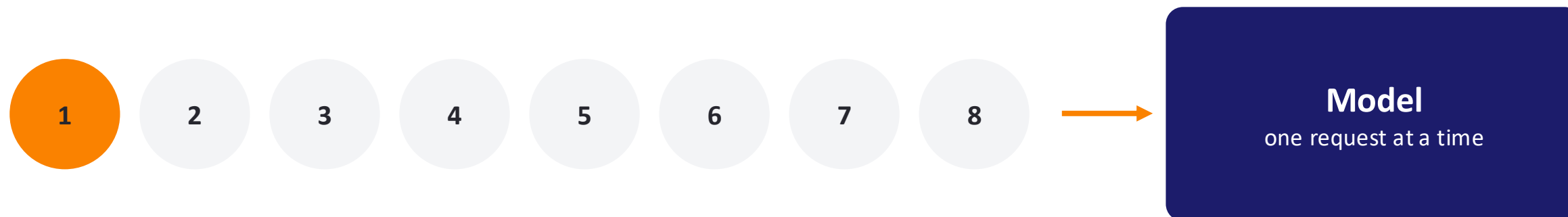
**11.73 s**

p95 response time, 8x slower

Same model, same tickets, same T4. The only change is how many people are asking. Measured 23 Sept.

DAY 2 | S8 WHAT BREAKS AT SCALE

**It degrades because a simple server answers requests one after another**



**Request 8 waits for the seven in front of it.**

The GPU is not full. It is idle between steps while the queue grows.

DAY 2 | S8 WHAT BREAKS AT SCALE

# Production serving batches requests continuously and pages the memory cache

1

## Continuous batching

New requests join the running batch as soon as any request finishes, so the GPU never waits for the slowest answer

2

## Paged memory cache

Each request's working memory is stored in small pages, so many more requests fit on one card

**Both are standard in open-source production servers. You change the server, not the model.**

Read: [Anyscale, continuous batching: anyscale.com/blog](https://www.anyscale.com/blog/continuous-batching) | [vLLM docs: docs.vllm.ai](https://docs.vllm.ai)

DAY 2 | S8 WHAT BREAKS AT SCALE

**Published result: roughly 44 times the output tokens per second at 64 concurrent users**

$\approx 44\times$

output tokens per second at 64 users

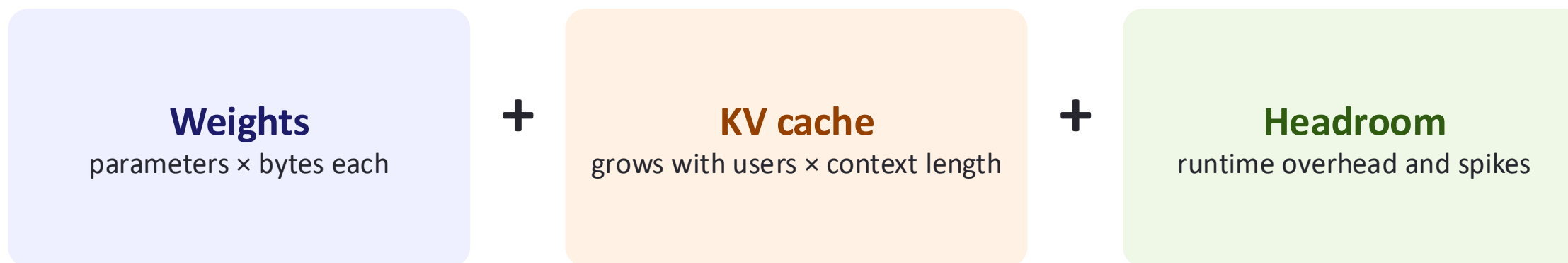
**What it compares**

vLLM, a production serving engine, against llama.cpp, the engine inside the Ollama server you ran this morning. One H200 GPU, Llama 3.1 8B, 1 to 64 users.

Source: Red Hat Developer, benchmark published September 2025 and repeated June 2026.

Source: [Red Hat Developer, vLLM or llama.cpp \(Sept 2025\)](#) | [Ollama vs vLLM \(Aug 2025\)](#)

## Memory arithmetic: weights plus cache plus headroom



**Worked example:** an 8-billion-parameter model at 16-bit is  $8B \times 2 \text{ bytes} = 16 \text{ GB}$  of weights, before a single user connects.  
Every concurrent user then adds cache on top.

Try it: [Hugging Face Model Memory Calculator: huggingface.co/spaces/hf-accellerate](https://huggingface.co/spaces/hf-accellerate)

## DAY 2 | S8 WHAT BREAKS AT SCALE

# A 24GB card serves small models; an 80GB card serves medium ones

| Model | Precision | Weights alone | 24GB card               | 80GB card               |
|-------|-----------|---------------|-------------------------|-------------------------|
| 8B    | 16-bit    | ≈ 16 GB       | Fits, little cache room | Fits, plenty of cache   |
| 8B    | 4-bit     | ≈ 4 to 5 GB   | Fits well               | Fits, many users        |
| 32B   | 4-bit     | ≈ 16 to 18 GB | Tight                   | Fits well               |
| 70B   | 16-bit    | ≈ 140 GB      | No                      | No, needs several cards |
| 70B   | 4-bit     | ≈ 35 to 40 GB | No                      | Fits, moderate cache    |

Weights only: parameters × bits ÷ 8. Cache and headroom come on top.

Read: [Hugging Face, Optimizing LLMs for speed and memory](#)

DAY 2 | S8 WHAT BREAKS AT SCALE | DECISION TABLE

# Quantization trades memory for quality: measure the trade on your own test set

| Precision | Bytes per parameter | When it wins                                     | What it costs                                                 |
|-----------|---------------------|--------------------------------------------------|---------------------------------------------------------------|
| 16-bit    | 2                   | The reference; plenty of GPU memory              | The most memory                                               |
| 8-bit     | 1                   | Halving memory with little visible change        | Usually small quality loss                                    |
| 4-bit     | About 0.5           | Fitting a larger model or many users on one card | Measurable loss, often on strict formats and harder reasoning |

**Run your eval at each precision before you choose. General benchmarks do not know your schema.**

Read: [Hugging Face quantization overview](#) | [QLoRA paper: arxiv.org/abs/2305.14314](#)

DAY 2 | S8 WHAT BREAKS AT SCALE

# Self-hosted cost per million tokens, against yesterday's API figure

## THE FORMULA

$$\text{cost per 1M tokens} = \text{GPU cost per hour} \div (\text{tokens per second} \times 3,600) \times 1,000,000$$

**Self-hosted, measured on a T4**

**≈ \$1.60**

per million output tokens, if busy 24 hours a day

**API, from yesterday**

**\$1.20 to \$50**

per million output tokens, efficient to frontier

Check: [Azure NVadsA10 v5 sizes: learn.microsoft.com](https://learn.microsoft.com/en-us/azure/virtual-machines/sizes/nvads-a10-v5)

DAY 2 | S8 WHAT BREAKS AT SCALE | WHAT GOES WRONG

## Sized for average load, the service desk assistant fails at month-end peak

### What happened

The assistant runs well for weeks. At month-end close, ticket volume jumps, the queue grows, and answers take minutes. Users go back to phoning.

### What caused it

Capacity was planned from the daily average, on a server that handles requests one after another.

### What catches it

A load test at peak concurrency before go-live, a batching server, and a queue limit that routes overflow to a fallback.

# Fine-tuning changes how a model answers, not what it knows

## Tuning reliably changes

- ✓ Output format and schema
- ✓ Which label it picks
- ✓ Tone and length

## Tuning does not reliably add

- ✗ New facts
- ✗ This month's procedure revision
- ✗ What a user may see

**OQ example:** tuning teaches a model to turn ticket INC-004412 into clean JSON. It does not teach it the current VPN policy.

## DAY 2 | S9 WHEN TO FINE-TUNE

# Retrieval wins most OQ cases, because OQ's knowledge keeps changing

| What OQ needs                            | Retrieval                              | Fine-tuning                      |
|------------------------------------------|----------------------------------------|----------------------------------|
| Procedures that get revised              | Re-index the new revision the same day | Retrain, re-test, redeploy       |
| Answers an auditor can check             | Cites the document and section         | No source to point to            |
| Users who see different documents        | Filter by permission at query time     | Every user gets the same weights |
| Rare questions nobody wrote examples for | Answers from any indexed page          | Guesses from what it saw         |

**OQ example:** "What is the gas test interval for confined space entry?" belongs to retrieval. The answer lives in a procedure that will be revised.

DAY 2 | S9 WHEN TO FINE-TUNE

# Tuning wins where the job is a pattern repeated at volume

1

## Fixed schema

Every output must parse, every time

**OOQ:** Drilling daily reports to NPT events for the dashboard

2

## House style

A consistent voice and structure

**OOQ:** Inspection findings written the same way, every time

3

## High-volume classification

Same decision, thousands of times

**OOQ:** Near-miss reports to hazard type and life-saving rule

4

## Shrink large into small

Teach a small model one job a large model does

**OOQ:** A frontier model labels 2,000 shift-log events; a small model on your own VM does it from then on

DAY 2 | S9 WHEN TO FINE-TUNE

# Service desk triage is a textbook tuning case

## IN: A REAL-LOOKING TICKET

INC-004412  
hi my laptop LAP-04412 wont connect to vpn since morning, tried restart already. urgent pls need Tavrona ERP for month end close



## OUT: THE SAME SEVEN FIELDS, EVERY TIME

```
{ "category": "network",  
  "affected_system": "VPN",  
  "asset_tag": "LAP-04412",  
  "urgency": "high", "impact": "single_user",  
  "requested_action": "Restore VPN access",  
  "routing_queue": "network" }
```

### High volume

Every ticket, every day

### Fixed schema

7 fields, 5 scored by exact match

### A cost argument

A small local model per ticket, not an API call

## DAY 2 | S9 WHEN TO FINE-TUNE | DECISION TABLE

# LoRA is the default for IT teams; full fine-tuning rarely pays

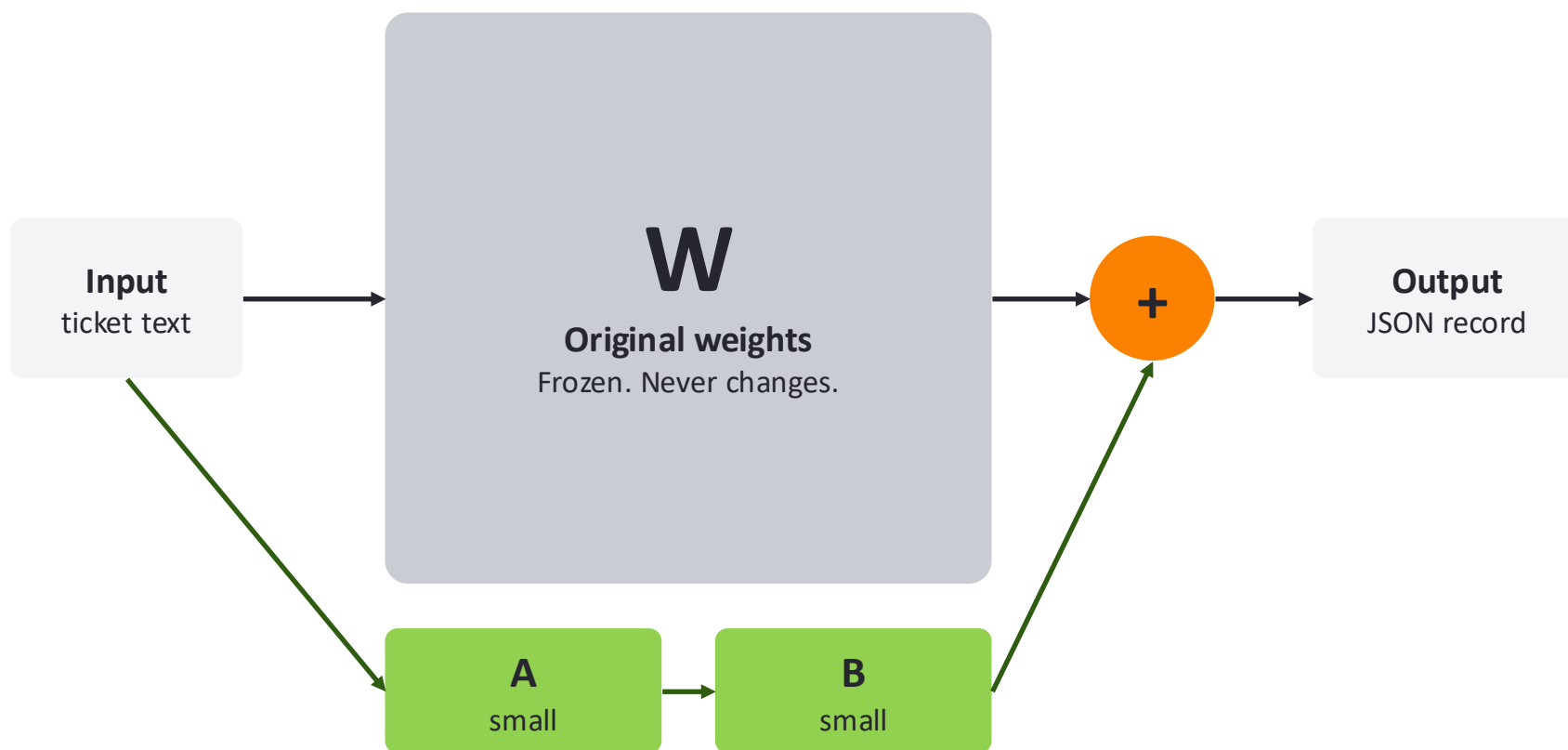
| Option         | When it wins                                 | What it costs                                                | OQ example                                |
|----------------|----------------------------------------------|--------------------------------------------------------------|-------------------------------------------|
| Full fine-tune | Deep behavior change with a large GPU budget | Multi-GPU training and a full model copy per version         | Rarely justified for an IT use case       |
| LoRA           | Most format and classification tasks         | One GPU; the adapter is a small file beside the base model   | Ticket triage on an OQ-hosted GPU VM      |
| QLoRA          | When GPU memory is the limit                 | Slightly slower training, small quality cost from 4-bit base | Tuning on a single 16GB card such as a T4 |

Sources: Hu et al., LoRA, 2021. Dettmers et al., QLoRA, 2023.

Read: [Hugging Face PEFT docs: huggingface.co/docs/peft](https://huggingface.co/docs/peft)

## DAY 2 | S9 WHEN TO FINE-TUNE

# LoRA trains two small adapters and leaves the model frozen



## 10,000×

fewer trainable parameters  
than a full fine-tune, in the  
original LoRA paper on a 175B  
model

**Today's adapter:** 11.3 million  
trainable parameters, 1.5% of the  
1B model

Hu et al., 2021

**DAY 2 | S9 WHEN TO FINE-TUNE**

# Dataset quality decides the outcome more than any setting does

| Problem in the data                   | What the model learns from it                                         |
|---------------------------------------|-----------------------------------------------------------------------|
| Duplicates and near-duplicates        | Overweights a few tickets; your scores look better than reality       |
| Validation rows leaking from training | A test it has already seen; accuracy you will never get in production |
| Schema violations in the labels       | Invalid output, learned on purpose                                    |
| Coverage gaps                         | Nothing about the ticket types you left out                           |
| Class imbalance left unexamined       | Rare categories such as ERP get routed wrong                          |

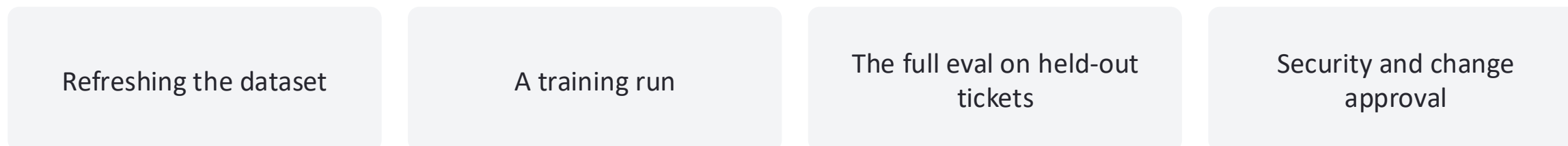
**In S10 you run these five checks on the ticket set before anything trains.**

DAY 2 | S9 WHEN TO FINE-TUNE

# Every base model change means retuning, and nobody budgets for it



## BUDGET EACH CYCLE FOR



**Hosted fine-tuning can disappear:** OpenAI closed it to new organizations in May 2026 and closes it to everyone on 6 January 2027. An adapter you own stays yours.

Source: [OpenAI deprecations: developers.openai.com](https://openai.com/deprecations)

## Tuned to teach facts, the model states them wrongly with confidence

### What happened

A team tunes on Q&A pairs from pump manuals. Asked for the seal flush pressure on P-1201A, the model answers in perfect format with the value for a different pump.

### What caused it

Each fact appeared in a few examples. The model learned the answer style strongly and the facts weakly, then blended similar pumps.

### What catches it

Score factual questions on held-out data against the source. Route lookups to retrieval, which cites the manual page.

DAY 2 | S9 WHEN TO FINE-TUNE

# Record your decision in your spec: tune or not, and why

## FOUR QUESTIONS

- 1 Is the output a fixed format that code will parse?
- 2 Is the volume high enough that cost per call matters?
- 3 Does everything the answer needs arrive in the input?
- 4 Can you build a few hundred clean labeled examples?

**Mostly yes: tune. Mostly no: retrieve or prompt.**

## WRITE THIS IN YOUR SPEC

### Tuning decision

**Decision:** tune / do not tune

**Because:** one sentence

**What would change it:** one condition

**Retune budget:** per base model change

## DAY 2 | S10 BUILD THE DATASET

# The target: seven fields, five of them scored by exact match

| Field            | Allowed values                                           | Scored by   |
|------------------|----------------------------------------------------------|-------------|
| category         | access, hardware, software, network, erp, telecom, other | Exact match |
| affected_system  | Text, or null                                            | Exact match |
| asset_tag        | Text, or null                                            | Exact match |
| urgency          | low, medium, high, critical                              | Exact match |
| impact           | single_user, team, site, enterprise                      | Exact match |
| requested_action | One short imperative clause                              | Rubric      |
| routing_queue    | Text                                                     | Rubric      |

Read: [JSON Schema: json-schema.org](https://json-schema.org)

DAY 2 | S10 BUILD THE DATASET | LAB

# Turn raw tickets into clean training pairs, and prove they are clean

NOTEBOOK

**04\_dataset\_builder**

TIME BOX

50 minutes, 40 in the notebook

RUNS ON

Colab, CPU runtime. No GPU needed

WORK IN

Your capstone group

IN AZURE

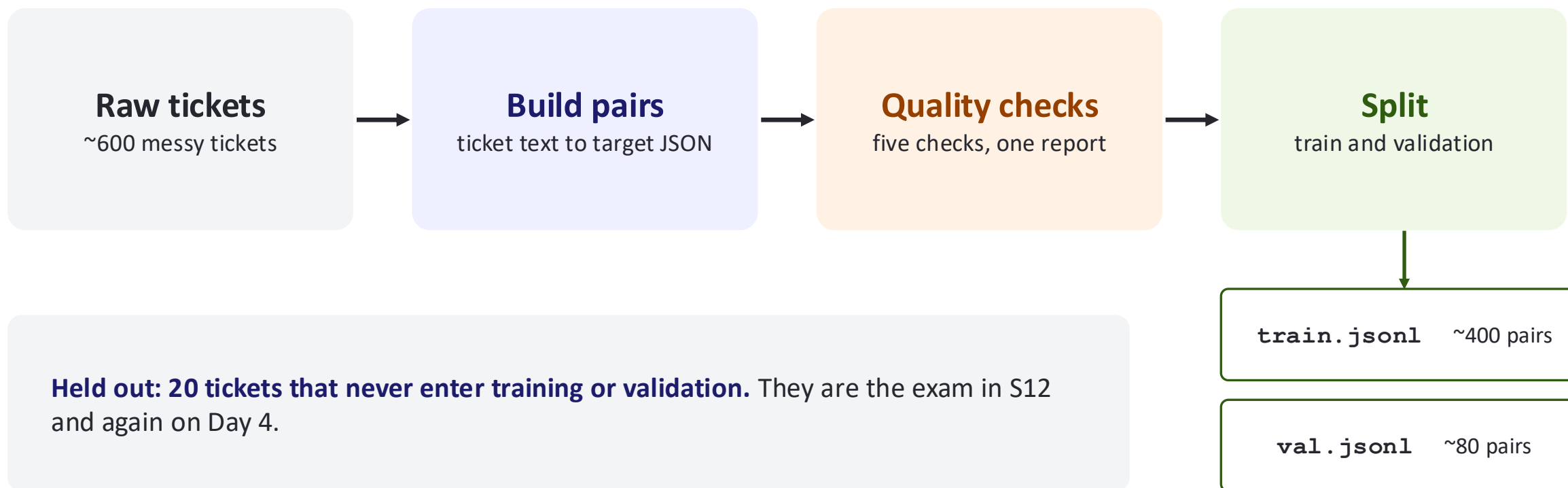
The same checks on an export of your own tickets, kept in your tenant

**Open in Colab:** link from the repo README



Scan to open the notebook

## You are building the pipeline between raw tickets and a training run



## Four steps, numbered to match the notebook's sections

1

### Load raw tickets

Read the ticket set and print the category counts

2

### Build pairs

Complete the TODO that maps ticket text to the target record

3

### Run the quality checks

Read every flag in the report before you fix anything

4

### Split

Write train and validation files, then re-run the leakage check

# Checkpoint: the report flags four kinds of problem in the ticket set

## YOU SHOULD NOW SEE

✓ Near-duplicate tickets

✓ Rows leaking into validation

✓ Schema violations

✓ Class imbalance

### WHAT THE CHECKS FIND IN THE TRAINING SET

|                   |          |      |
|-------------------|----------|------|
| near-duplicates   | 20 pairs | FAIL |
| leaked into val   | 5 rows   | FAIL |
| schema violations | 10 rows  | FAIL |
| class imbalance   | reported | WARN |

after cleaning  
train 373 rows  
val 72 rows

Imbalance is reported, never corrected:  
that decision is yours.

## If you are stuck, load the saved dataset and keep going

### The saved result

Pre-built, already cleaned:

`data/finetune/train.jsonl`

`data/finetune/val.jsonl`

Facilitator report: `facilitator/prebaked_outputs/`

### The one-line fallback

Point the S11 notebook at the saved files. You lose nothing for this afternoon, and the report still makes sense to read.

DAY 2 | S10 BUILD THE DATASET | DEBRIEF

The data problem you catch late is the one that costs most

Which problem would have hurt most if you had not caught it?

**This feeds S11 at 12:45.** Your train and validation files are what the model learns from after lunch.

## Schema or fact: sort these in 90 seconds

- 1 I read out eight OQ requests, one at a time
- 2 Stand up if the request is mostly about format. Stay seated if it is mostly about facts
- 3 Where the room splits, we stop and argue for 30 seconds

**Why this one:** The same sort decides tune or retrieve for your own system.

DAY 2 | S11 FINE-TUNE IT | LAB

# Fine-tune a small model to turn tickets into the seven-field record

NOTEBOOK

**05\_finetune**

TIME BOX

90 minutes. Training takes about 7 on a T4

RUNS ON

Colab, T4 GPU runtime

IF NO T4

Pre-baked adapter, slide 50

IN AZURE

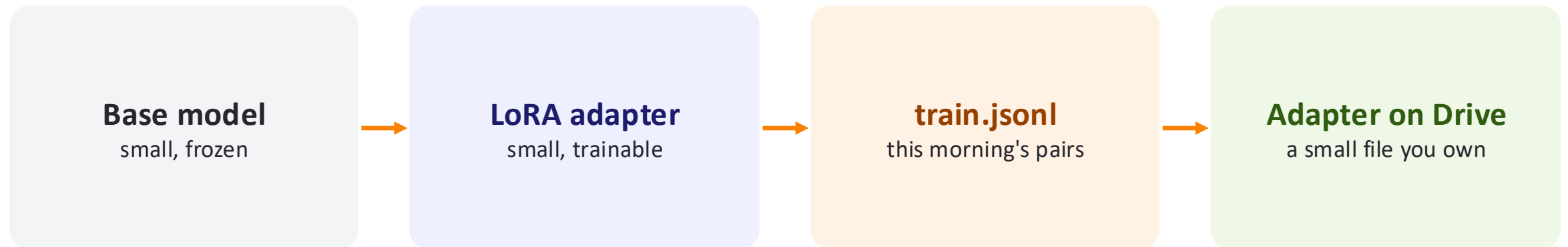
Fine-tuning in Microsoft Foundry. Training runs in US or EU regions only



Scan to open

Help: [Google Colab FAQ, GPU limits: research.google.com/colaboratory](https://research.google.com/colaboratory/)

## You are training an adapter, not a new model



**Saved to Drive as it trains.** A disconnect costs the current step, not the run.

## Four steps, then the model trains while you write the rubric

1

### Load the base model

The small model named in the notebook header

2

### Attach the adapter

Read the adapter settings; do not change them today

3

### Train

Start it, check the first loss values, then leave it

4

### Save and disconnect

Adapter to Drive, then release the GPU

## While it trains, agree how your group will score the output

| Field                      | How it is scored                            |
|----------------------------|---------------------------------------------|
| category, urgency, impact  | Exact match against the label               |
| affected_system, asset_tag | Exact match, and null when absent           |
| requested_action           | Your group's rubric: correct, partly, wrong |
| routing_queue              | Exact match or rubric: CONFIRM              |

### Argue about urgency

"Urgent pls, month end" from one user. Is that high or medium? Agree a rule and write it down.

DAY 2 | S11 FINE-TUNE IT | LAB

# Checkpoint: a falling loss curve and an adapter saved to Drive

YOU SHOULD NOW SEE

✓ Loss falling, then flattening

✓ Training finished without errors

✓ Adapter folder on Drive

✓ GPU runtime disconnected

## MEASURED ON A COLAB T4, 23 SEPT

llama3.2 1B, LoRA rank 16, 400 training tickets

steps 72 / 72 in 6.5 minutes

training loss 0.742 -> 0.021

validation 0.137 -> 0.055 -> 0.047

peak GPU memory 7.5 GB allocated

adapter 11.3M parameters, on Drive

check 5/5 schema-valid after reload

## If training fails or runs out of time, use the pre-baked adapter

### The saved result

`checkpoints/adapter_prebaked/`

Trained on the same data, same settings

### The one-line fallback

Load the pre-baked adapter in S12 instead of yours. Every comparison still works, and you can rerun training tonight.

DAY 2 | S12 DID IT WORK | LAB

# Score the base model and your tuned model on 20 tickets neither has seen

NOTEBOOK

**06\_compare\_base\_tuned**

TIME BOX

45 minutes, 25 in the notebook

RUNS ON

Colab. Loads your adapter from Drive

WORK IN

Your capstone group

IN AZURE

Foundry evaluations, or this same script pointed at your Azure endpoint



Scan to open

DAY 2 | S12 DID IT WORK | LAB

# Checkpoint: one table, base against tuned, field by field

YOU SHOULD NOW SEE

✓ Schema validity, base and tuned

✓ Exact match per field

✓ Your rubric score for requested\_action

## THE REFERENCE RUN, 20 HELD-OUT TICKETS

|                  | base 1B     | tuned       |
|------------------|-------------|-------------|
| schema-valid     | 12/20       | 20/20       |
| category         | 8/20        | 16/20       |
| routing_queue    | 7/20        | 16/20       |
| requested_action | 1/20        | 16/20       |
| <b>urgency</b>   | <b>4/20</b> | <b>7/20</b> |
| all six fields   | 2/20        | 4/20        |
| invented values  | 3           | 0           |

The base column moves by 1 or 2 between runs.  
The tuned column is stable.

## DAY 2 | S12 DID IT WORK

# Read the failures out loud: where tuning helped and where it did not

| Field            | Base  | Tuned | What it tells you                                                                 |
|------------------|-------|-------|-----------------------------------------------------------------------------------|
| Valid JSON       | 12/20 | 20/20 | Format is what tuning fixes best                                                  |
| category         | 8/20  | 16/20 | Our house labels, learned from examples                                           |
| urgency          | 4/20  | 7/20  | A judgment field: still wrong most of the time, so an agent always sets it        |
| asset_tag        | 15/20 | 20/20 | The base copied the example tag from the prompt; the tuned model reads the ticket |
| requested_action | 1/20  | 16/20 | House phrasing learned, scored by word overlap                                    |

DAY 2 | S12 DID IT WORK

# A tuned model can be confidently wrong in new ways

## HELD-OUT TICKET

**INC-005370**

Subject: new phone need the authenticator app setup

I got a new phone and the authenticator app is not set up on it, so I cannot approve any sign-in. It is only me, my colleagues are fine... I suspect a virus.

Correct: access, urgency high, queue identity\_access

## TUNED OUTPUT

### TUNED MODEL, VALID JSON

```
{  
  "category": "hardware",  
  "affected_system": null,  
  "asset_tag": null,  
  "urgency": "medium",  
  "impact": "single_user",  
  "requested_action": "Install the user's device  
authenticator app",  
  "routing_queue": "end_user_computing"  
}
```

WRONG

WRONG

WRONG

**Valid format makes a wrong answer look finished. That is the danger.**

DAY 2 | S12 DID IT WORK

# Owning a tuned model: where the weights live, who may retune, how you roll back

1

## Where it lives

Adapter and base model versioned together, in a store you control

2

## Who may retune

A named owner, with the dataset and eval that approve each version

3

## How you roll back

Swap back to the previous adapter file. Minutes, not a rebuild

DAY 2 | S12 DID IT WORK | DEBRIEF

## Tuning won on format; Day 4 shows whether it wins anything else

Where did tuning help, and where did it not?

**This feeds Day 4, S19.** Retrieval joins the same table, on the same 20 tickets.

# The tuned model that quietly broke the NPT dashboard

## WHAT HAPPENED

A small tuned model turns drilling daily reports into NPT events for the digital drilling dashboard. It scored well on its test set and ran for six months. Then a new drilling contractor started on two rigs, and the NPT code list gained three codes. Every record still passed the schema. At the quarterly review, "waiting on equipment" hours had fallen by a third, and nobody could explain why.

## IN YOUR GROUP

- 1 Why did the dashboard change when nobody changed the model?
- 2 What would have caught it before the quarterly review?
- 3 Who owns the retrain, and what number starts it?

DAY 2 | CLOSE | SPEC CHECKPOINT

# Spec v0.3 tonight: four sections get today's numbers

| Section             | What goes in                                                       | From today         | The Azure line                               |
|---------------------|--------------------------------------------------------------------|--------------------|----------------------------------------------|
| 2 The decision      | Tune or not, and why: your four answers                            | S9, slide 36       | Fine-tuning in Foundry, or your own adapter? |
| 4 Model and serving | Model, server and version; the endpoint name                       | S7, your endpoint  | Which VM or endpoint, which region           |
| 6 Quality           | Base against tuned, field by field; what an agent must check       | S12, your table    | Foundry evaluations, or this script          |
| 7 Cost and capacity | p95 at your load, capacity at your target, cost per million tokens | S8, your load test | The same test on your Azure endpoint         |
| Save                | Spec_<name>_v0.3.docx, posted in the group                         | Tonight            | Tomorrow 08:00: two minutes each             |

DAY 2 | CLOSE

## Today you served, stressed and tuned a model of your own

**S7** Your own model endpoint

**S8** Load numbers for your cost model

**S10** A clean ticket dataset

**S11** A tuned adapter, saved to Drive

**S12** Base against tuned, one table

**Artifact 7**  
in progress

Retrieval joins the comparison on Day 4.

DAY 2 | CLOSE

# Tomorrow: ground it. Bring the retrieval problem your system has to solve

## WHAT WE BUILD

A retrieval pipeline over manuals and procedures

Hybrid search that finds tags like P-1201A

Extraction from diagrams and scanned pages

## HAVE READY AT 08:00

✓ Colab open and signed in

✓ Today's files saved to Drive

✓ One sentence: the retrieval problem in your system